A MINIMAL FIELD HANDBOOK

The Minimal Open-Weights Handbook

Download, quantize, serve and fine-tune open models on your own hardware.

Mosafer

llama.cpp · Ollama · vLLM · TRL

V1.0 — COMPACT EDITION

NOVEMBER 2026

Contents

1 Orientation **3**
- 1.1 The license spectrum (read before you build) . . . 3
- 1.2 The ecosystem map . . . 3
- 1.3 Hardware reality . . . 4

2 Getting the Weights **5**
- 2.1 Downloading . . . 5
- 2.2 What is in a checkpoint . . . 5
- 2.3 GGUF vs. safetensors . . . 5

3 Local Inference **7**
- 3.1 Ollama: the fastest start . . . 7
- 3.2 llama.cpp: the engine room . . . 7
- 3.3 transformers: when you need the graph . . . 7
- 3.4 Memory: weights plus KV cache . . . 8

4 Quantization **9**
- 4.1 The format zoo . . . 9
- 4.2 Reading a quant name . . . 9
- 4.3 Sizing a model to a machine . . . 9
- 4.4 Quantizing a model you have . . . 9

5 Serving: an OpenAI-Compatible API **11**
- 5.1 vLLM: throughput serving . . . 11
- 5.2 Ollama and llama.cpp servers . . . 11
- 5.3 The client side . . . 11
- 5.4 Structured output . . . 12

6 Fine-Tuning: QLoRA End to End **13**
- 6.1 The dataset: a list of chat messages . . . 13
- 6.2 The training script . . . 13
- 6.3 The hyperparameters that matter . . . 14

7 Merging, Converting, Shipping **15**
- 7.1 Merge the adapter . . . 15
- 7.2 To GGUF and quantize . . . 15
- 7.3 The Ollama wrapper . . . 15
- 7.4 Upload . . . 16

8 Evaluation and Hygiene **17**
- 8.1 Standard benchmarks . . . 17
- 8.2 Your own eval set beats the public one . . . 17
- 8.3 Before/after for a fine-tune . . . 17
- 8.4 Hygiene checklist . . . 17

A Cheat Sheet **19**
A.1 Weights 19
A.2 Inference 19
A.3 Serving (OpenAI-compatible) 19
A.4 Fine-tune to ship 20
A.5 Evaluation 20
A.6 Memory quick table 20

CHAPTER 1

Orientation

"Open weight" means the trained parameters are downloadable — not that the data, code, or license are open. This handbook is the practical loop over that landscape: fetch weights, run them locally at usable speed, expose them behind an API, fine-tune them on one GPU, convert and quantize the result, and evaluate it honestly.

1.1 The license spectrum (read before you build)

License family	Examples and implications
Apache-2.0 / MIT	`Qwen`, `Mistral`, `SmolLM2`, `Gemma` (terms) — commercial use, fine-tune, redistribute: the safe default
Community licenses	`Llama` (acceptable-use + 700M MAU clause), `Falcon` — fine for almost everyone; read before shipping a product
Research-only	some `ALMA`, `BLOOM` variants — non-commercial; never in a product

Derivative work inherits the base license: a LoRA on a Llama model carries Llama's terms; a LoRA on Qwen is Apache. When a paper or product depends on a checkpoint, record the model id *and* license in the README (Git handbook, Ch. 8).

1.2 The ecosystem map

Tool	Layer	Role
huggingface_hf CLI	weights	download/cache any checkpoint
`safetensors`	weights	the standard weight format (safe, mmap-able)
`GGUF` + `llama.cpp`	weights + inference	quantized single-file models; CPU/CPU+GPU; laptops
`Ollama`	inference	model manager + server over llama.cpp — the easiest start
`vLLM`	serving	high-throughput GPU server, OpenAI-compatible API
transformers / trl	training	load, fine-tune (the HF handbook, Ch. 2–7)
`lm-eval-harness`	evaluation	standard academic task suites

```
pip install transformers datasets trl peft accelerate
pip install vllm                      # Linux + CUDA; large install
brew install ollama                   # macOS; Linux: curl script
```

1.3 Hardware reality

Model	bf16	4-bit	Comfortable on
0.5–2B	1–4 GB	0.5–1.5 GB	CPU / any laptop
7–9B	14–18 GB	4–5.5 GB	12 GB GPU or 16 GB RAM (CPU, slow)
13–14B	26–28 GB	7–9 GB	16–24 GB GPU
70B	~140 GB	~40 GB	multi-GPU or 2×24 GB

Weights are most of the memory at rest; the KV cache (Ch. 3) grows with context length and batch on top. Quantization (Ch. 4) trades a little quality for a lot of memory — it is the default for local work.

> **PITFALL**
>
> Bigger is not automatically better *for your task*: a 1.5B instruct model with the right prompt beats a mistuned 14B base model, and domain LoRAs on small models outrank generic large ones. Choose by license + context length + measured task quality (Ch. 8), not by parameter count alone.

CHAPTER 2

Getting the Weights

Everything starts at the Hub. This chapter: downloading cleanly, what the files are, and how to keep a machine of caches sane.

2.1 Downloading

```
hf download Qwen/Qwen2.5-1.5B-Instruct            # whole repo -> cache
hf download Qwen/Qwen2.5-1.5B-Instruct --local-dir ./qwen2.5-1.5b
hf download Qwen/Qwen2.5-1.5B-Instruct \
    config.json model.safetensors tokenizer.json   # selected files only
```

(The CLI was `huggingface-cli download`; `hf download` is the current spelling, both work.) Everything lands in `~/.cache/huggingface/hub/` unless `-local-dir` says otherwise; downloads are resumable and verified. Behind a slow or blocked route, point the client at a mirror: `export HF_ENDPOINT=https://hf-mirror.com`.

```
export HF_HUB_OFFLINE=1        # after downloading: run from cache only
```

2.2 What is in a checkpoint

File	Role
`config.json`	architecture + generation defaults — defines the model
`model.safetensors` or shards	the weights (safetensors: no pickle, mmap-safe)
`tokenizer.json`, `tokenizer_config.json`	the vocabulary and rules
`generation_config.json`	sampling defaults for `generate()`
`model-0000X-of-0000N.safetensors` + `.index.json`	big models, sharded
`*.gguf`	single-file quantized format for llama.cpp (Ch. 4)

`safetensors` replaced `.bin` everywhere because it cannot execute embedded code (the pickle hazard) and memory-maps instantly.

2.3 GGUF vs. safetensors

Two formats, two worlds:

- **safetensors** — the training/transformers/vLLM world: full precision (bf16/fp16) or served-quantized (Ch. 5).
- **GGUF** — the llama.cpp/Ollama world: one self-describing file with quantization baked in (`Q4_K_M`, `Q8_0`, ...), runs on CPU+GPU hybrids, macOS Metal, and old cards.

Many popular checkpoints ship official GGUF siblings (`bartowski/Qwen2.5-1.5B-Instruct-GGUF`, per-quant files). The full route from your own fine-tune to GGUF is Ch. 7.

PITFALL

Disk drift is the silent space killer: three experiments later, `~/.cache/huggingface/hub` holds 300 GB of half-used models. Audit with `du -sh ~/.cache/huggingface/hub/*` and delete by folder — the cache is fully re-downloadable, and `HF_HUB_OFFLINE=1` breaks the moment you delete a model still in use, so keep the id list in the repo README.

GATED MODELS

Llama and Gemma require accepting terms on the Hub, then an access token: `hf auth login`, once per machine. The token is per-user, not per-project — keep it out of scripts and never commit it (Git handbook, Ch. 1's pitfall applies double here).

CHAPTER 3

Local Inference

Three runtimes, three tradeoffs: `transformers` (flexible, research-shaped), `Ollama` (zero-friction chat + API), `llama.cpp` (the engine under Ollama — maximal control, runs everywhere). This chapter gets all three talking.

3.1 Ollama: the fastest start

```
ollama pull qwen2.5:1.5b-instruct
ollama run qwen2.5:1.5b-instruct        # interactive chat
ollama serve                            # API on localhost:11434 (auto-runs)
ollama list                             # what is on disk
```

That is a working local LLM: model management, chat, and a REST API (Ch. 5) with one command per step. `Ollama` picks a sensible quantization per model and handles GPU/CPU offload automatically.

3.2 llama.cpp: the engine room

```
git clone https://github.com/ggerganov/llama.cpp && cd llama.cpp
cmake -B build && cmake --build build -j          # GGML_CUDA=on for NVIDIA

llama-cli -m qwen2.5-1.5b-instruct-q4_k_m.gguf -cnv \
    --n-gpu-layers 99                             # offload all layers
llama-bench -m model.gguf                          # tokens/sec benchmark
```

`-ngl 99` ships every layer to the GPU; lower numbers split layers between GPU and CPU — the dial for running 14B on a 8 GB card. Speed scales with memory bandwidth: expect 20–60 tok/s for 1.5–8B Q4 models on modern laptops, single digits on pure CPU for 14B+.

3.3 transformers: when you need the graph

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

name = "Qwen/Qwen2.5-1.5B-Instruct"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(
    name, torch_dtype=torch.bfloat16, device_map="auto")

messages = [{"role": "user", "content": "Say hi in five words."}]
text = tok.apply_chat_template(messages, tokenize=False,
                               add_generation_prompt=True)
inputs = tok(text, return_tensors="pt").to(model.device)
out = model.generate(**inputs, max_new_tokens=32,
                     pad_token_id=tok.eos_token_id)
print(tok.decode(out[0][inputs["input_ids"].shape[1]:],
```

```
                skip_special_tokens=True))
```

Use this when the task needs logits, per-token scores, custom sampling loops, or training (HF handbook, Ch. 6); use Ollama/llama.cpp when the task is *chat and serving*.

3.4 Memory: weights plus KV cache

The KV cache grows with `context_length * batch` and can exceed the weights at long contexts:

Dial	Effect
`-c 4096` (llama.cpp) / `-max-model-len` (vLLM)	cap context: the biggest memory lever
`GQA` models (Qwen, Llama 3)	small KV per token — long context is cheap on modern checkpoints
`-n-gpu-layers`	split layers GPU/CPU on small cards
`-flash-attn`	faster attention, less memory (GPU builds)

PITFALL

The chat template is not optional: llama.cpp's `-cnv` applies the model's own template; Ollama ships it in the model card; in transformers you call `apply_chat_template` (Ch. 6, HF handbook). Hand-rolling prompt markup on an instruct model is the classic "local model is dumb" bug — the model is fine, the wrapper is wrong.

CHAPTER 4

Quantization

Quantization stores weights in fewer bits — 16 \to 4 is a 4x memory cut for a small quality cost. It is what makes local LLMs normal-person hardware. This chapter covers the formats and how to choose.

4.1 The format zoo

Format	Runtime	Notes
GGUF Q4_K_M	llama.cpp / Ollama	the local default; mixed 4-bit scheme, quality near Q8
GGUF Q8_0	llama.cpp / Ollama	near-lossless, 8.5 bits/weight — when RAM allows
bitsandbytes NF4	transformers	on-the-fly 4-bit; the QLoRA base (HF handbook, Ch. 7)
AWQ / GPTQ	vLLM, GPU servers	4-bit, calibrated on data; fast GPU serving
fp8 / bf16	vLLM (H100+)	datacenter serving; no local concern

4.2 Reading a quant name

Q4_K_M = 4-bit, K-quant blocks, _M = medium mix (more bits for attention and sensitive layers). Practical ladder: Q2_K (small, noticeably dumber) \to Q4_K_M (the sweet spot) \to Q5_K_M \to Q8_0 (near-lossless) \to F16. A 7–8B model at Q4_K_M is \(\sim \)4.5 GB; the same model at bf16 is 15 GB.

4.3 Sizing a model to a machine

```
bits = {"Q2_K": 3.4, "Q4_K_M": 4.8, "Q8_0": 8.5, "bf16": 16}
for q, bpw in bits.items():   # footprint: bpw * params / 8, + KV cache
    print(f"7.6B model @ {q:7s}: {bpw * 7.6 / 8:5.1f} GB + KV cache")
```

Leave 1–2 GB headroom for the KV cache at 4–8k context and the OS. The empirical check is `llama-bench` (Ch. 3) on *your* machine — published tok/s numbers do not transfer across hardware.

4.4 Quantizing a model you have

```
python convert_hf_to_gguf.py out/merged --outtype f16 \
    --outfile merged-f16.gguf
./build/bin/llama-quantize merged-f16.gguf merged-q4km.gguf Q4_K_M
llama-cli -m merged-q4km.gguf -cnv        # sanity check before shipping
```

Always sanity-check a fresh quant on ten prompts before deploying — a broken quantization shows up as fluent gibberish, which no metric in your pipeline will catch as loudly as reading one output.

> **PITFALL**
> Quality loss is task-dependent and non-uniform: 4-bit barely moves chat quality, but small-model Q2_K quantization measurably hurts math and code generation. Evaluate the quant on your task (Ch. 8) if the

application is accuracy-sensitive, and keep the model card's recommended quant — bartowski-style tables list per-quant perplexity deltas.

THE ONE-LINE DECISION

Laptop / no GPU: GGUF `Q4_K_M` via Ollama. One 12–24 GB GPU + serving traffic: vLLM with AWQ or bf16 (Ch. 5). Training the base: bf16 or NF4 via bitsandbytes (Ch. 6). Everything else is detail.

CHAPTER 5

Serving: an OpenAI-Compatible API

Every serious local runtime exposes the same wire protocol: the OpenAI Chat Completions API. Serve once, and every client—scripts, LangChain, your editor, the `openai` package—speaks to your own weights.

5.1 vLLM: throughput serving

```
vllm serve Qwen/Qwen2.5-1.5B-Instruct --max-model-len 8192 \
    --port 8000
# production flavors: --quantization awq, --tensor-parallel-size 2,
#                     --max-num-seqs 64
```

vLLM's engine batches concurrent requests continuously (paged KV cache) — single requests feel the same as transformers, but tens of concurrent users cost far less than 10x the GPU time. It is the default for anything with more than one user.

5.2 Ollama and llama.cpp servers

```
ollama serve                          # API on :11434/v1
llama-server -m model.gguf --port 8080 -c 8192
```

Both expose OpenAI-compatible endpoints with the same shapes; llama.cpp adds `/health` and per-slot controls, Ollama adds model management. Use Ollama for personal tools, vLLM when traffic or latency SLAs are real.

5.3 The client side

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1",
                api_key="not-needed")        # local servers accept any key
resp = client.chat.completions.create(
    model="Qwen/Qwen2.5-1.5B-Instruct",      # vLLM: the served name
    messages=[{"role": "user",
               "content": "Summarize the KV cache in one sentence."}],
    temperature=0.7, max_tokens=128)
print(resp.choices[0].message.content)

stream = client.chat.completions.create(
    model="Qwen/Qwen2.5-1.5B-Instruct", messages=[{
        "role": "user", "content": "Count to five."}],
    stream=True)
for chunk in stream:
    print(chunk.choices[0].delta.content or "", end="", flush=True)
```

Point `base_url` at `localhost:11434/v1` (Ollama) or `localhost:8080/v1` (llama.cpp) and the identical code runs — the server is swappable behind an unchanged client.

5.4 Structured output

Local serving makes constrained decoding a flag, not a project:

```
resp = client.chat.completions.create(
    model="Qwen/Qwen2.5-1.5B-Instruct",
    messages=[{"role": "user", "content":
               "Extract {name, year} from this abstract: ..."}],
    response_format={"type": "json_object"},   # vLLM: guided JSON
    extra_body={"guided_json": {
        "type": "object",
        "properties": {"name": {"type": "string"},
                       "year": {"type": "integer"}},
        "required": ["name", "year"]}},
)
```

vLLM's guided decoding guarantees schema-valid JSON via constrained sampling — the difference between a demo and a pipeline.

PITFALL

Three serving gotchas: (1) the `model` field must match what the server loaded (vLLM echoes the served id; Ollama wants the tag `qwen2.5:1.5b-instruct`); (2) `max_model_len` bounds prompt + completion — long-RAG requests fail with cryptic 400s when the cap is hit, not with a warning; (3) GPU memory for concurrent batching: vLLM pre-allocates a large fraction of VRAM for KV cache (`-gpu-memory-utilization 0.9` default) — on a shared card, lower it explicitly.

LOCAL OR CLOUD?

The API contract is identical, so the deployment decision is a config line: privacy and cost favor local; frontier quality and scale favor hosted. Building against `base_url` from day one keeps both doors open — that single line is the whole lock-in question.

CHAPTER 6

Fine-Tuning: QLoRA End to End

One GPU, one evening, a model that speaks your domain. QLoRA — 4-bit frozen base + LoRA adapters (HF handbook, Ch. 7) — is the standard recipe; this chapter runs it end to end with `trl`.

6.1 The dataset: a list of chat messages

The format that matters is the chat template, not a magic schema:

```
import json

rows = [{"messages": [
            {"role": "system", "content": "You are a materials science assistant."},
            {"role": "user", "content": "Why does annealing soften a metal?"},
            {"role": "assistant", "content": "It removes dislocations..."}]},
        # ... 500-5000 of these, real and diverse
       ]
json.dump(rows, open("data/train.json", "w"), indent=1)
```

Quality beats quantity: 500 clean, varied, correctly-templated examples out-teach 50k scraped ones. Build from your domain's real Q/A pairs; deduplicate; hold out 5% as an eval slice.

6.2 The training script

```
from datasets import load_dataset
from peft import LoraConfig
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from trl import SFTConfig, SFTTrainer
import torch

name = "Qwen/Qwen2.5-1.5B-Instruct"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(
    name,
    quantization_config=BitsAndBytesConfig(
        load_in_4bit=True, bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16),
    device_map={"": 0})

peft_cfg = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                      target_modules="all-linear",
                      task_type="CAUSAL_LM")
cfg = SFTConfig(
    output_dir="out/qwen-materials",
    per_device_train_batch_size=2,
    gradient_accumulation_steps=16,       # effective batch 32
    learning_rate=2e-4, num_train_epochs=3,
    lr_scheduler_type="cosine", warmup_ratio=0.03,
    bf16=True, logging_steps=10,
```

```
    max_seq_length=1024,
)
trainer = SFTTrainer(model=model, args=cfg, train_dataset=ds,
                     processing_class=tok, peft_config=peft_cfg)
trainer.train()
trainer.save_model("out/qwen-materials-lora")   # adapters, ~50-200 MB
```

`SFTTrainer` applies the model's chat template to the `messages` column automatically — the same template the serving stack will use, which is exactly why the fine-tune transfers. A 1.5B QLoRA trains in under an hour on a 12 GB card; 7–8B fits a 24 GB card with the same script.

6.3 The hyperparameters that matter

Knob	Sensible value and why
`r=16, alpha=32`	capacity/ratio; raise `r` only if underfitting is measured, not felt
`learning_rate=2e-4`	LoRA tolerates 10–100x full-finetune LRs; 1e-4–3e-4 is the band
`epochs=1-3`	more epochs on small data memorizes; watch eval loss, stop at the bend
`max_seq_length=1024`	cap at your real prompt+answer length; memory scales with it
`gradient_accumulation`	effective batch 16–32 stabilizes LoRA steps

PITFALL

The signature QLoRA failures: (1) `CUDA out of memory` at a long batch — lower `max_seq_length` or batch, not the quant; (2) adapters trained with a *different* template than inference uses — the model works at temperature 0.01 and nowhere else; always let `SFTTrainer`/`apply_chat_template` own the format; (3) catastrophic tone drift — 3 epochs on 300 examples overfits into parroting; eval on held-out prompts (Ch. 8) every epoch via `eval_strategy="steps"`.

UNSLOTH AND FRIENDS

`unsloth` reimplements the LoRA kernels for 2x speed and half the memory with a near-identical API — a drop-in win for single-GPU runs. `axolotl` wraps the same recipe in YAML for people who prefer config files. Underneath, it is all the Ch. 7 (HF handbook) mechanics shown here.

CHAPTER 7

Merging, Converting, Shipping

Training produced a 200 MB adapter. This chapter turns it into something usable everywhere: merged into the base weights, converted to GGUF, quantized, wrapped for Ollama, and uploaded.

7.1 Merge the adapter

```
import torch
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer

name = "Qwen/Qwen2.5-1.5B-Instruct"
base = AutoModelForCausalLM.from_pretrained(name, torch_dtype=torch.bfloat16)
model = PeftModel.from_pretrained(base, "out/qwen-materials-lora")
model = model.merge_and_unload()          # bake BA into W
model.save_pretrained("out/qwen-materials-merged")
tok = AutoTokenizer.from_pretrained(name)
tok.save_pretrained("out/qwen-materials-merged")
```

The merged folder is a plain checkpoint: loadable by transformers and vLLM like anything on the Hub. Keep the adapter file too — small adapters are versionable artifacts; merged models are 3–30 GB build outputs (Git handbook Ch. 1's rule: outputs are rebuilt, not committed).

7.2 To GGUF and quantize

```
git clone https://github.com/ggerganov/llama.cpp && cd llama.cpp
pip install -r requirements.txt
python convert_hf_to_gguf.py ../out/qwen-materials-merged \
    --outfile qwen-materials-f16.gguf --outtype f16
./build/bin/llama-quantize qwen-materials-f16.gguf \
    qwen-materials-q4km.gguf Q4_K_M
llama-cli -m qwen-materials-q4km.gguf -cnv      # read its answers!
```

`convert_hf_to_gguf.py` reads the merged checkpoint's config and emits an F16 GGUF; `llama-quantize` then produces every smaller format from that one file.

7.3 The Ollama wrapper

```
cat > Modelfile <<'EOF'
FROM ./qwen-materials-q4km.gguf
PARAMETER temperature 0.7
SYSTEM "You are a materials science assistant. Answer concisely."
EOF
ollama create qwen-materials -f Modelfile
ollama run qwen-materials "Why does annealing soften a metal?"
```

One `Modelfile` makes the fine-tune a first-class citizen of the Ollama world — chat, API (Ch. 5), and one-command distribution to any colleague's machine.

7.4 Upload

```
hf auth login
hf upload mosafer/qwen2.5-1.5b-materials out/qwen-materials-merged .
hf upload mosafer/qwen2.5-1.5b-materials-lora out/qwen-materials-lora .
```

Then edit the auto-generated model card: the base model id, the dataset provenance, the license (inherited from the base — Ch. 1), the training config, and three example prompts. A model card without the template and hyperparameters is a binary blob, not a model.

PITFALL

The merge-order trap: quantizing the *adapter-side* files (QLoRA's 4-bit base) produces garbage — always `merge_and_unload` into bf16/f16 *first*, quantize the merged weights after. And keep base+adapter versions locked together: an adapter trained on `Qwen2.5-1.5B-Instruct` will not load on `Qwen2.5-3B` — `PeftModel.from_pretrained` checks and refuses, which is the good outcome.

THE SHIPPING CHECKLIST

Merged bf16 checkpoint + adapters + GGUF (one quant) + Modelfile + model card with base id, data provenance, license, and three example prompts. That bundle runs on a datacenter GPU, a laptop, and a reviewer's machine — which is the whole point of open weights.

CHAPTER 8

Evaluation and Hygiene

"It feels smarter" is not a measurement. This chapter: standard benchmarks via `lm-eval-harness`, task-specific eval sets, and the hygiene rules that keep both honest (the Statistics handbook is the companion volume).

8.1 Standard benchmarks

```
pip install lm-eval
lm_eval --model hf \
    --model_args pretrained=Qwen/Qwen2.5-1.5B-Instruct,dtype=bfloat16 \
    --tasks mmlu,gsm8k,hellaswag --batch_size auto \
    --output_path results/qwen1.5b/
```

One command, standard tasks, comparable numbers. For local GGUF models, `-model gguf` works; for served models, `-model local-completions` with an endpoint. Record the exact command and harness version next to the results — task prompts change between versions and silently shift scores.

8.2 Your own eval set beats the public one

Twenty problems from *your* domain, with reference answers, plus a grading script — that is an eval set, and it out-predicts MMLU for your use case:

```
import json, subprocess

cases = json.load(open("evals/materials_20.json"))
for c in cases:
    prompt = tok.apply_chat_template([{"role": "user",
        "content": c["q"]}], tokenize=False, add_generation_prompt=True)
    out = query_local(prompt)              # Ch. 5 client call
    c["answer"], c["pass"] = out, c["check"](out)
print(f"{sum(c['pass'] for c in cases)}/{len(cases)}")
```

Grade with exact match where possible, keyword checks where needed, and an LLM judge only where unavoidable — judge outputs are noisy and the judge's own biases leak in.

8.3 Before/after for a fine-tune

The only evaluation that justifies Ch. 6: same eval set, base model vs. fine-tuned model, same sampling settings. Ch. 8 of the Statistics handbook is the reporting template — binomial intervals over items, paired comparison across the eval set, and the delta with its interval. A fine-tune that moves MMLU but not your task eval set moved nothing that matters.

8.4 Hygiene checklist

- Record: model id + revision, quant, template, sampling params, harness version — one config block per eval run (Statistics handbook Ch. 1).

- Contamination: public benchmarks are *in* the training data of frontier open models; treat MMLU scores as saturation indicators, not capability proofs — your private eval set is the real number.
- Version pinning: `llama.cpp` and `vLLM` releases change numerics and defaults; pin the version next to the results.
- Reproducibility: temperature 0 (greedy) for eval runs, seeds for anything sampled, and the same offline flags (`HF_HUB_OFFLINE=1`) on every machine.

PITFALL

The leaderboard trap: choosing a base model from a public leaderboard without running your own eval set. Rankings aggregate tasks you do not have, at contexts you do not use, with prompts you would never write. The shortlist comes from leaderboards; the decision comes from your twenty problems.

THE LOOP, ASSEMBLED

Download (Ch. 2) `\(\to \)` serve (Ch. 5) `\(\to \)` eval (chapter 8) `\(\to \)` fine-tune (Ch. 6) `\(\to \)` merge + quantize (Ch. 7) `\(\to \)` serve `\(\to \)` eval `\(\to \)` ship. Every stage produces a recorded artifact; the delta between eval runs is the project. That loop, at whatever scale your hardware allows, is what working with open weights means.

APPENDIX A

Cheat Sheet

A.1 Weights

Task	Code
Download	`hf download Qwen/Qwen2.5-1.5B-Instruct` (+ `--local-dir`)
Selected files	`hf download <id> config.json model.safetensors`
Offline	`export HF_HUB_OFFLINE=1`
Mirror	`export HF_ENDPOINT=https://hf-mirror.com`
Auth (gated)	`hf auth login`

A.2 Inference

Task	Code
Ollama	`ollama pull qwen2.5:1.5b-instruct` \(\to\) `ollama run ...` / `ollama serve`
llama.cpp	`llama-cli -m m.gguf -cnv -n-gpu-layers 99`
Server	`llama-server -m m.gguf --port 8080 -c 8192`
Benchmark	`llama-bench -m m.gguf`
transformers	`AutoModelForCausalLM.from_pretrained(name, torch_dtype=torch.bfloat16, device_map="auto")`
Memory dial	context cap `-c` / `--max-model-len`; `-n-gpu-layers` split

A.3 Serving (OpenAI-compatible)

Task	Code
vLLM	`vllm serve <id> --max-model-len 8192 --port 8000`
Client	`OpenAI(base_url="http://localhost:8000/v1", api_key="x")`
Chat	`client.chat.completions.create(model=..., messages=[...])`
Stream	`... stream=True` + loop over `chunk.choices`
JSON mode	`response_format=\{"type": "json_object"\}` / `extra_body=\{"guided_json": schema\}`

A.4 Fine-tune to ship

Task	Code
QLoRA	NF4 `BitsAndBytesConfig` + `LoraConfig(r=16, lora_alpha=32, target_modules="all-linear")`
Train	`SFTTrainer(model, args=SFTConfig(...), peft_config=...)`
Merge	`PeftModel.from_pretrained(base, adapter)` \(\to\) `merge_and_unload()`
GGUF	`convert_hf_to_gguf.py merged --outtype f16` \(\to\) `llama-quantize ... Q4_K_M`
Ollama	`Modelfile`: `FROM ./m.gguf` + SYSTEM \(\to\) `ollama create name -f Modelfile`
Upload	`hf upload <org/name> <dir> .` + model card

A.5 Evaluation

Task	Code
Harness	`lm_eval --model hf --model_args pretrained=<id> --tasks mmlu,gsm8k`
Own set	20 domain cases + `check()` grading function
Compare	base vs. fine-tune, same sampling, delta with interval (Statistics handbook Ch. 8)

A.6 Memory quick table

Model	bf16	Q4_K_M	runs on
0.5–2B	1–4 GB	0.5–1.5 GB	CPU / laptop
7–9B	14–18 GB	4–5.5 GB	12 GB GPU
13–14B	26–28 GB	7–9 GB	16–24 GB GPU
70B	~140 GB	~40 GB	multi-GPU

THE HABITS THAT MATTER

Pin the model id, revision, quant, template, and sampling params in every experiment record. Let the chat template own the prompt. Sanity check every fresh quant on ten prompts. Evaluate on your own twenty problems, with intervals. Ship adapters + merged bf16 + one GGUF. That is the whole discipline of working with open weights.